



Evolutionary Action Selection for Gradient-Based Policy Learning

Yan Ma¹, Tianxing Liu¹, Bingsheng Wei¹, Yi Liu¹, Kang Xu¹, and Wei Li^{1,2}(✉)

¹ Academy for Engineering and Technology, Fudan University, Shanghai, China
fd_liwei@fudan.edu.cn

² Ji Hua Laboratory, Foshan, China

Abstract. Evolutionary Algorithms (EAs) and Deep Reinforcement Learning (DRL) have recently been integrated to take advantage of both methods for better exploration and exploitation. The evolutionary part of these hybrid methods maintains a population of policy networks. However, existing methods focus on optimizing the parameters of policy network, which is usually high-dimensional and tricky for EA. In this paper, we shift the target of evolution from high-dimensional parameter space to low-dimensional action space. We propose Evolutionary Action Selection-Twin Delayed Deep Deterministic Policy Gradient (EAS-TD3), a novel hybrid method of EA and DRL. In EAS, we focus on optimizing the action chosen by the policy network and attempt to obtain high-quality actions to promote policy learning through an evolutionary algorithm. We conduct several experiments on challenging continuous control tasks. The result shows that EAS-TD3 shows superior performance over other state-of-art methods.

Keywords: Evolutionary Algorithm · Reinforcement Learning · Policy Search

1 Introduction

Deep Reinforcement Learning (DRL) has achieved impressive performance in Go [28], Atari games [19], and continuous control tasks [10, 15]. The purpose of DRL is to train an optimal or nearly-optimal policy network that maximizes the reward function or other user-provided reinforcement signal. Recently, Evolutionary Algorithms (EA) have been applied to search the parameter space of neural networks to train policies and showed competitive results as DRL [24, 30]. Some work compares DRL and EA [16, 25], highlighting their respective pros and cons. However, an emerging research direction tends to integrate them to promote each other for better policy search [23, 27], in that they have several complementary properties and can be integrated to benefit from the best of both worlds. As one of the pioneers, [13] proposed Evolutionary Reinforcement Learning (ERL), which has shown promising results in continuous control tasks. ERL maintains a population containing n policy networks trained by EA and one policy network trained by DRL. The policy population provides a large number of samples

(s, a, r, s') to the RL policy, and RL policy will be injected into the population periodically to replace the poorer individual. With the help of EA’s global optimization ability, ERL explores the parameter space of the policy network to search for outstanding solutions. Most variants of the ERL [2, 12, 17, 22] follow the parameter space of the policy network as the target of evolution. However, taking the policy network parameters as the evolution target may pose a potential problem. It is well known that EA is deficient in optimizing high-dimensional spaces. Although EA can be leveraged to train policy networks, it suffers from low sample efficiency since it is gradient-free. Corresponding to ERL methods, the contribution of the evolutionary part will be limited. In particular, we find that the evolutionary part may weaken the performance of the overall method at high dimensions of the parameter space. As illustrated in Fig. 1c (see more details in Sect. 4.1), the performance of ERL method decays with the increase of policy parameters. The problem in ERL methods is attributed to the use of EA to train the strategy network directly. In a nutshell, taking the parameter space of the policy network as the target of evolution will limit the contribution of EA to such hybrid methods. The motivation of this work is to transfer the evolutionary target from the high dimensional parameter space to a low dimensional space where EA is more proficient and avoids the problem mentioned earlier to serve the best of both worlds better.

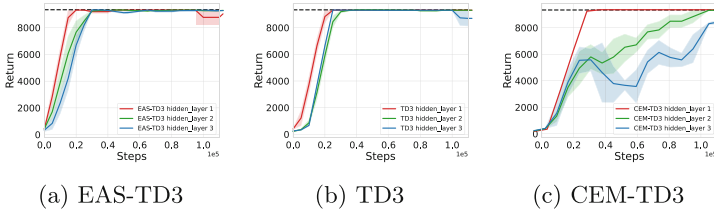


Fig. 1. Comparative Experiment of EAS-TD3 (our work), CEM-TD3 (one of the sota methods in ERL variants), TD3 on Inverted Double Pendulum task from Gym [3] with different policy network parameters. The performance of CEM-TD3 decreases while that of EAS-TD3 remains stable.

Our goal is unambiguous, which aims to train an excellent policy to maximize the cumulative reward of the given task. A good policy $\mu(a|s)$ means it can choose a good action a according to the state s to maximize the cumulative reward. The quality of the action largely determines the quality of the policy. More importantly, the action space is relatively low dimensional and generally will not change for a given task. Thus, we intend to take the action space as the target of evolution. Concretely speaking, we leverage EA to optimize the action chosen by the RL policy and obtain better evolutionary actions. The policy network trained by RL collects samples in the environment. EA extracts actions from samples and evolves better evolutionary actions to promote the RL policy learning. It maintains the flow of information between the EA part and RL part.

Based on the above insight, we propose Evolutionary Action Selection (EAS). EAS utilizes actions selected by RL policy to form a population and utilizes Particle Swarm Optimization (PSO) [11] to evolve the action population from generation to generation. Finally, we obtain better evolutionary actions, which can promote the learning process through the evolutionary action gradient. We choose PSO for two reasons. Firstly, it has been widely used in many fields and proved effective in practice [32]. Secondly, it is easy to follow and will not add too much computational burden. In addition, PSO can be replaced by other evolutionary algorithms, such as genetic algorithm [18], cross-entropy method [7].

Our contributions are threefold: (1) We empirically demonstrate that taking the policy network parameter space as the evolutionary target may lead to performance degradation. (2) We transfer the target of evolution from high-dimensional parameter space to low-dimensional action space and propose a simple and effective mechanism to evolve action. (3) We apply EAS to TD3 [8] as EAS-TD3 and conduct a series of empirical studies on a benchmark suite of continuous control tasks to prove the feasibility and superiority of our approach.

2 Related Work

The idea of incorporating learning with evolution has been around for many years [1, 9]. With the brilliance of reinforcement learning, recent literature [5, 17] has begun to revisit the combination of the two to improve the performance of the overall approach.

As mentioned earlier, this paper is related to the recently proposed Evolutionary Reinforcement Learning (ERL) [13] framework. ERL combines Genetic Algorithm (GA) [18] with the off-policy DRL algorithm (DDPG) [4] and incorporates the two processes to run concurrently, formulating a framework. Specifically, ERL maintains a policy population trained by EA and a policy network trained by RL. By maintaining interactive information flow between EA and RL, the performance of the overall method is promoted. The ERL framework has triggered various variants, making the efficient combination of EA and RL an emerging research direction for both the EA and RL community. Collaborative Evolutionary Reinforcement Learning (CERL) [12] is the follow-up work of ERL. CERL attempts to train multiple policy networks with different hyperparameters to address the DRL’s sensitivity to hyperparameters. Moreover, Proximal Distilled Evolutionary Reinforcement Learning (PDERL) [2] attempts to figure out the catastrophic forgetting of the neural network caused by the genetic operator used in ERL. CEM-RL [22] removes the single policy network trained by RL and instead allows half of the policy networks in the population to be trained directly by RL and the other half by cross entropy method (CEM) [7]. This approach magnifies the impact of gradient-based policy learning methods on the evolutionary population, which improves the sample efficiency of the ERL framework. AES-RL [14] proposes an efficient asynchronous method for integrating evolutionary and gradient-based policy search, which shortens the training time. QD-PG [21] introduces Quality-Diversity (QD) algorithm for RL to address the problem of deceptive reward.

Our approach focuses on integrating the advantages of EA and DRL to learn policies better. We shift the target of evolution from high-dimensional policy network parameter space to low-dimensional action space and empirically demonstrate that evolving the action is a favorable alternative to evolving the parameters of the policy network.

3 Methodology

3.1 Evolutionary Action Selection (EAS)

Taking the critic network Q_{μ_θ} of TD3 as the fitness evaluator, EAS follows the process of PSO and evolves the action a chosen by the current policy network $\mu_\theta(s)$. EAS's output is the evolutionary action a^e with a higher Q value than action a . The pseudocode of EAS is shown in Algorithm 1.

In EAS, we first add Gaussian noise to the action a and make it an action set $\mathbb{A}$ containing multiple noisy actions, which serves as the initial population. The fitness of each action is its Q value generated by the critic network Q_{μ_θ} . Secondly, we initialize the velocity vector, which determines the direction and step length when updating actions in the population. Then, we initialize the personal best action set $\mathcal{P} = (p_1^b \dots p_n^b \dots p_N^b)$, which records the best solution of each action in the population found so far. The best action in $\mathcal{P}$ is called the global best action g^b , which represents the best solution found so far. In each generation, action a_n^t in the population will be evaluated by fitness evaluator Q_{μ_θ} to obtain the fitness Q_n^t . Based on the magnitude of Q_n^t , we update the personal and global best action. Moreover, the velocity is updated by Eq. 2, which subsequently will be used to update actions and get the next generation of population. In Eq. 2, inertia weight ω describes previous velocity's influence on current velocity. Acceleration coefficients c_1 and c_2 represent the acceleration weights toward the personal best action and the global best action. r_1 and r_2 are random variables uniformly distributed in $[0, 1]$. Through several iterations, we can obtain the global best action g^b with the highest Q value searched so far and denote it as the evolutionary action a^e . The relationship of Q value between a^e and a is:

$$Q_{\mu_\theta}(s, a^e) \geq Q_{\mu_\theta}(s, a) \quad (1)$$

which reveals that EAS can increase the Q value of the action so that the action will have a higher expected reward. We claim the changing from a to a^e is the action evolution. The evolutionary action a^e is better than a and has a higher expected reward.

3.2 EAS-TD3 Framework

Figure 2 illustrates a diagram of EAS-TD3. At each timestep t , the policy observes s_t and outputs action a_t . Then, we receive a reward r_t and environment transitions to next state s_{t+1} . These four elements make up the sample

Algorithm 1. Evolutionary Action Selection**Input:** State s , action a , critic network Q_{μ_θ} **PSO parameters:** Inertia weight ω , acceleration coefficients c_1, c_2 , random coefficients r_1, r_2 **Output:** Evolutionary action a^e

```

1: Extend the action  $a$  with Gaussian noise  $\epsilon$  to form the initial action population
    $\mathbb{A} = (a_1 \dots a_n \dots a_N)$ ,  $a_n = a + \epsilon_n, \epsilon \in \mathcal{N}(0, \sigma)$ ,  $N$  is the number of actions
2: Initialize the velocity of action  $\mathcal{V} = (v_1 \dots v_n \dots v_N)$ ,  $|v_n| = |a_n| = D$ ,  $v_n \in [-v_{max}^D, v_{max}^D]$ 
3: Initialize personal best action set  $\mathcal{P} = (p_1^b \dots p_n^b \dots p_N^b)$  and global best action  $g^b$ 
4: for  $t = 1$  to  $T$  do
5:   for  $n = 1$  to  $N$  do
6:      $Q_n^t \leftarrow Q_{\mu_\theta}(s, a_n^t)$  ▷ Evaluate the action
7:      $p_n^b \leftarrow a_n^t$ ; if  $Q_n^t > Q_{p_n^b}$  ▷ Update the personal best action
8:      $g^b \leftarrow p_n^b$ ; if  $Q_{p_n^b} > Q_{g^b}$  ▷ Update the global best action
9:      $v_n^{t+1} = \omega * v_n^t + c_1 * r_1 * (p_n^b - a_n^t) + c_2 * r_2 * (g^b - a_n^t)$  (2) ▷ Update the velocity
10:     $a_n^{t+1} = a_n^t + v_n^{t+1}$  (3) ▷ Update the action
11:   end for
12: end for
13: Obtain the global best action  $g^b$ , representing evolutionary action  $a^e$ 

```

(s_t, a_t, r_t, s_{t+1}) , which will be stored into the replay buffer R . Then, EAS performs evolution on action a to obtain evolutionary action a^e , which will be stored into an archive $\mathcal{A}$. We draw the same mini-batches from $\mathcal{R}$ and $\mathcal{A}$ and update the current policy μ_θ with deterministic policy gradient [29] and evolutionary action gradient (will be described below). EAS adopts the way of delayed policy updates: one policy update for two Q function updates, which is the same as TD3.

Through EAS, we obtain evolutionary actions, storing their corresponding state-action pairs (s, a^e) into an archive $\mathcal{A}$. As mentioned in Sect. 3.1, the evolutionary action has a higher expected reward than the original action. Consequently, we intend to make the action space of RL policy similar to the space of evolutionary actions so that the evolutionary actions can contribute to the RL policy learning. Due to the idea given above, we construct a loss as below:

$$L_{evo}(\theta, \mathcal{A}) = \mathbb{E}_{(s_i, a_i^e) \sim \mathcal{A}} \left[\|\mu_\theta(s_i) - a_i^e\|^2 \right] \quad (4)$$

where s_i and a_i^e represent the state and evolutionary action sampled from $\mathcal{A}$ respectively, and θ represents the learning parameters in RL policy μ_θ . We call L_{evo} *evolutionary action gradient*. Furthermore, we weigh the constructed loss with an extra Q_{filter} as proposed in [20]:

$$Q_{filter} = \begin{cases} 1, & \text{if } Q_{\mu_\theta}(s_i, a_i^e) > Q_{\mu_\theta}(s_i, \mu_\theta(s_i)), \\ 0, & \text{else.} \end{cases} \quad (5)$$

The purpose of Q_{filter} is to drop out $L_{evo}(\theta, \mathcal{A})$ when the action chosen by the current RL policy is superior to the evolutionary actions in archive $\mathcal{A}$. The reason is that if an evolutionary action generated long ago is sampled to update parameters, the action chosen by the current policy may be better than the previous evolutionary action. Thus, we need to filter out those outdated evolutionary actions. We periodically draw a batch of state-action pairs from $\mathcal{A}$ and utilize Eq.6 to update parameters.

$$\nabla_{\theta} L_{Q_{filter}evo} = Q_{filter} \nabla_{\theta} L_{evo} \quad (6)$$

EAS promotes the evolution of actions, which can be used to guide strategy learning. The introduction of Q_{filter} effectively avoids a poor direction for learning when the action chosen by the current policy is superior to the previous evolutionary action in the archive. We refer to the dual integration with TD3 as Evolutionary Action Selection-Twin Delayed DDPG (EAS-TD3).

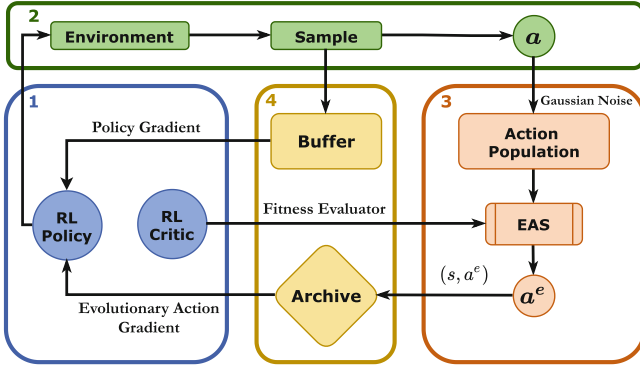


Fig. 2. A high-level view of EAS-TD3. RL policy interacts with the environment to generate the sample (s, a, r, s') , which will be stored in a replay buffer. Action a is added with Gaussian noise to form an evolutionary population. Then use EAS to update the action population to obtain the evolutionary action a^e . The state-action pair (s, a^e) will be stored in an archive. We draw batches from the replay buffer and archive to update the current RL policy with policy gradient and evolutionary action gradient.

4 Experiment

The main purpose of this section is to investigate the mechanism of EAS and the performance of EAS-TD3 compared to other evolutionary reinforcement learning methods.

4.1 A Toy Example

As mentioned in Sect. 1, ERL methods generally choose the high-dimensional parameter space as the target of evolution, which is tricky for EA to optimize and may lead to the collapse performance of the overall approach. To confirm this, we set up a toy example. Specifically, we increase the policy network’s hidden layer and conduct experiments on the Inverted Double Pendulum from OpenAI gym. As a relatively easy continuous control task for most modern algorithms, Inverted Double Pendulum has an 11-dimensional state space and 1-dimensional action space.

Figure 1 shows the comparative performance of EAS-TD3, CEM-TD3, TD3. As shown in Fig. 1c, the performance of CEM-TD3 decays with increasing policy parameters. With the increase of hidden layers, the parameter space dimension that needs to be optimized increases dramatically. It makes it difficult for CEM to search for decent parameters of the policy network, which attenuates the performance of policy population and the contribution of the evolutionary part. Then, the stagnant policy population produces worthless samples to store in the replay buffer. The RL part may be drowning in these useless samples. In short, taking the policy network parameter space as the evolutionary target may lead to performance degradation. Correspondingly, the performance of EAS-TD3 is not affected as shown in Fig. 1a, since its evolutionary part optimizes the low-dimensional action space, which is rarely changed for a given task.

4.2 Experimental Setup

We select four continuous control locomotion tasks from OpenAI Gym simulated by MuJoCo [31]: HalfCheetah-v3, Walker2d-v3, Ant-v3, Humanoid-v3. We present the average reward and the associated standard deviation over ten runs. We test the learning policy for each run on ten evaluation episodes every 5000 steps. In all figures of learning curves, unless specified otherwise, the x-axis represents the number of steps performed in the environment, and the y-axis represents the mean return obtained by the policy.

By default, all the hyperparameters for TD3 are the same as those in the original paper [8]. Here, we only provide the unique hyper-parameters of the EAS-TD3. With regard to PSO, inertia weight ω is 1.2, acceleration coefficients c_1, c_2 are both 1.5, the number of iterations T is 10, v_{max} is 0.1, random coefficients r_1, r_2 are random numbers from 0 to 1. The size of archive $\mathcal{A}$ is 100,000 for all environments. The action population size is 10. Our method is compared against the official implementations for TD3 [26], CEM-TD3 [22], CERL [12], PDERL [2], ERL [13]. These baselines contain a series of studies on the combination of EA and RL. The population size of all baselines is set to 10.

4.3 Experiments on MuJoCo Environments

Figure 3 demonstrates the reward curves in MuJoCo. In general, EAS-TD3 performs consistently well across most MuJoCo environments, indicating that EAS plays a significant role in the process of policy learning.

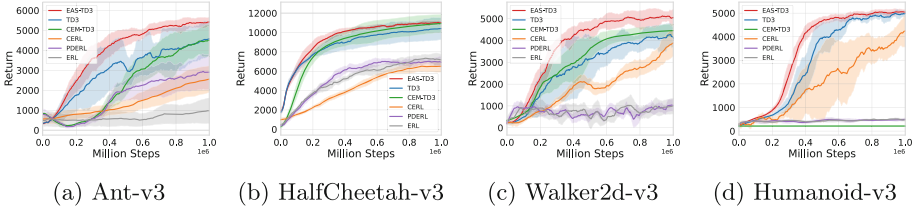


Fig. 3. The learning curves in MuJoCo environments. The shaded area represents mean $\pm$ standard deviation over the 10 runs.

Compared to ERL Methods. EAS-TD3 performs a considerable improvement on environments with high-dimensional state and action space like Humanoid, Ant, Walker2d. Why are ERL methods not performing as well as EAS-TD3 on these tasks? Note that the parameters of the policy network are associated with the state and action dimensions of the environment. For environments with higher dimensions, their policy networks also have more parameters, and more timesteps are needed for EA to search for sound policies. For example, a policy network in Humanoid-v3 task (state space 376 dims & action space 17 dims) consisting of a hidden layer with 400 and 300 nodes will have more than 270,000 parameters, which takes more timesteps to learn a decent policy for gradient-free evolutionary methods. Therefore, in high-dimensional tasks like Humanoid and Ant, the contribution of the evolutionary part in ERL methods will be weakened, resulting in the reduction of learning efficiency or even complete failure, which is consistent with the performance shown in the toy example. However, EAS-TD3 shifts the target of evolution from high-dimensional parameter space to low-dimensional action space to avoid the disaster of parameter dimension growth. The evolutionary part in EAS-TD3 focuses on evolving better actions and promoting policy learning through evolutionary action gradients. It is the main reason why EAS-TD3 performs better. Besides, CEM-TD3 performs better on HalfCheetah, probably because CEM can search for decent policies on these simple tasks. Moreover, we remark that the evolutionary part may even hinder the early learning of RL part in ERL methods. It is reflected in Ant and Walker2d tasks, where CEM-TD3, the best performer among the ERL methods, has a slightly slower learning efficiency than TD3. The reason may be attributed to the lackluster performance of the evolution population, which brings a vast amount of worthless experiences to the replay buffer, thus hindering the learning process of RL part. EAS-TD3 has no such concern since the evolution part does not directly train the policy network.

Compared to TD3. EAS-TD3 outperforms TD3 more or less in all environments. Compared with TD3, the increment of EAS-TD3 only lies in the influence of the evolutionary action gradient, which indicates the performance enhancement does come from the introduction of EAS. The evolutionary action can promote policy learning within different tasks. In HalfCheetah, EAS-TD3 achieves a minor improvement in experiments with multiple random seeds. Since

TD3 has already shown promising results in this environment, the evolutionary action can slightly enhance the sampling efficiency and performance. At the same time, EAS-TD3 performs well in high-dimensional complex tasks such as Ant and Humanoid, as the gradient of evolutionary action rapidly drives the policy’s action space toward regions with higher expected reward and significantly improves learning speed and final performance. We will discuss more how EAS works in the following section.

4.4 Ablation Studies

The Size of Archive $\mathcal{A}$. The capacity of archive $\mathcal{A}$ is generally not significant in that the policy will continue to learn while evolutionary action selection is based on the policy at that time. Therefore, we should control the size of $\mathcal{A}$ to keep evolutionary actions fresh. Otherwise, the outdated evolutionary actions will not play a role in guiding policy learning. We select different archive sizes (10,000, 50,000, 100,000, 500,000) and perform

an ablation study on the size of $\mathcal{A}$. As shown in Fig. 4, the archive size affects the performance. $\mathcal{A}$ stores state-action pairs (s, a^e) corresponding to evolutionary actions. As discussed earlier, since the evolutionary actions need to keep fresh, the archive size should not be too large. Otherwise, the archive $\mathcal{A}$ will store outdated actions, which may weaken the role of evolutionary action. In contrast, it should not be too small, or the diversity of actions in $\mathcal{A}$ may be poor. It will lead to the frequent sampling of limited evolutionary actions and hinder the exploration ability of policy. From Fig. 4, 100,000 seems to be a reasonable size for these tasks.

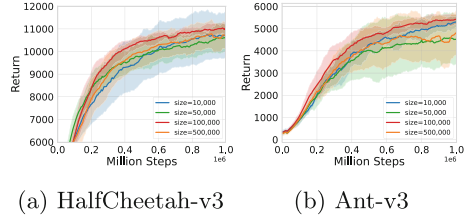


Fig. 4. Ablation study (mean $\pm$ standard deviation) on the size of archive $\mathcal{A}$. 100,000 seems to be a reasonable size.

4.5 Evolutionary Action Evaluation

In this section, we will investigate the mechanism of EAS and mainly answer a question: how do evolutionary actions promote policy learning?

Figure 5 shows the first two-dimensional distributions of actions chosen by the policy and the corresponding evolutionary actions during the training process of Walker2d-v3. Each plot demonstrates the distribution of all actions in every 100,000 timesteps. The performance of the policy will gradually improve over timesteps. Therefore, the action distribution plot of each column in Fig. 5 indicates a higher expected reward than the previous column. From Fig. 5, we can see that evolutionary actions predict the action space with higher expected reward in advance and guide the current policy to move towards there, which may explain why EAS works. For example, within the 0 to 100,000 timesteps (corresponding to the two plots in the first column), the distribution of the

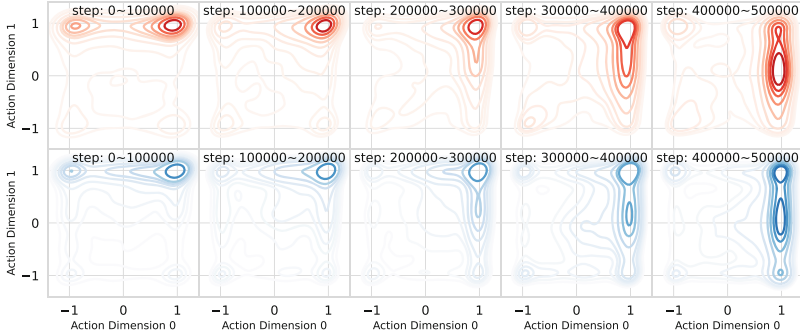


Fig. 5. The distribution of the first two action dimensions during the training process. The red contour represents the current policy’s actions and the blue contour represents the corresponding evolutionary actions. The performance of the policy will gradually improve over timesteps. The action distribution plot of each column indicates a higher expected reward than the previous column. Evolutionary actions predict the action space with a higher expected reward in advance and guide the current policy to move towards there. (Color figure online)

evolutionary action exhibits a trend from the upper right downward. The downward trend subsequently becomes more pronounced. Correspondingly, the policy action distribution shows the same trend under the guidance of both the policy gradient and evolutionary action gradient. It indicates that the evolutionary action gradient promotes strategy learning in the same direction as the policy gradient, which significantly accelerates the progress of policy learning. From Fig. 5, we also discover that the space of evolutionary actions does not deviate too much from the current policy’s action space, which is served as a gentle mentor and guides the current policy step by step. In conclusion, with the aid of evolutionary action gradient, we can make good use of evolutionary actions and finally play a role in the learning process.

5 Conclusion and Further Work

This paper proposes Evolutionary Action Selection (EAS) which applies evolution to the action chosen by the policy. We integrate EAS into TD3 as EAS-TD3, which is a novel hybrid method of EA and RL. Compared with other hybrid methods, our approach transforms the target of evolution from the high-dimensional parameter space to the low-dimensional action space. With the promotion of evolutionary actions, we train policies that substantially outperform those trained directly by TD3 and other compared methods. Extensive experiments and empirical analyses demonstrate that evolving action is a favorable alternative to evolving the parameters of the policy network. We believe that our work can trigger follow-up studies to address several interesting open questions. In future work, we can integrate the idea of Quality-Diversity (QD) [6], which evolves not only high-quality actions but also evolve various actions to deal with the environment with deceptive rewards.

Acknowledgments. This work was supported in part by Shanghai Municipal Science and Technology Major Project (No. 2021SHZDZX0103), in part by Ji Hua Laboratory (No. X190011TB190), in part by the Shanghai Engineering Research Center of AI and Robotics, and in part by the Engineering Research Center of AI and Robotics, Ministry of Education, China.

References

1. Ackley, D.: Interactions between learning and evolution. *Artif. Life II* **10**, 487–509 (1992)
2. Bodnar, C., Day, B., Lió, P.: Proximal distilled evolutionary reinforcement learning. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, pp. 3283–3290 (2020)
3. Brockman, G., et al.: OpenAI gym. arXiv preprint [arXiv:1606.01540](https://arxiv.org/abs/1606.01540) (2016)
4. Casas, N.: Deep deterministic policy gradient for urban traffic light control. arXiv preprint [arXiv:1703.09035](https://arxiv.org/abs/1703.09035) (2017)
5. Colas, C., Sigaud, O., Oudeyer, P.Y.: GEP-PG: decoupling exploration and exploitation in deep reinforcement learning algorithms. In: *International Conference on Machine Learning*, pp. 1039–1048. PMLR (2018)
6. Cully, A., Clune, J., Tarapore, D., Mouret, J.B.: Robots that can adapt like animals. *Nature* **521**(7553), 503–507 (2015)
7. De Boer, P.T., Kroese, D.P., Mannor, S., Rubinstein, R.Y.: A tutorial on the cross-entropy method. *Ann. Oper. Res.* **134**(1), 19–67 (2005). <https://doi.org/10.1007/s10479-005-5724-z>
8. Fujimoto, S., Hoof, H., Meger, D.: Addressing function approximation error in actor-critic methods. In: *International Conference on Machine Learning*, pp. 1587–1596. PMLR (2018)
9. Grefenstette, J.J., Moriarty, D.E., Schultz, A.C.: Evolutionary algorithms for reinforcement learning. arXiv e-prints, p. arXiv-1106 (2011)
10. Haarnoja, T., et al.: Soft actor-critic algorithms and applications. arXiv preprint [arXiv:1812.05905](https://arxiv.org/abs/1812.05905) (2018)
11. Kennedy, J., Eberhart, R.: Particle swarm optimization. In: *Proceedings of ICNN'95-International Conference on Neural Networks*, vol. 4, pp. 1942–1948. IEEE (1995)
12. Khadka, S., et al.: Collaborative evolutionary reinforcement learning. In: *International Conference on Machine Learning*, pp. 3341–3350. PMLR (2019)
13. Khadka, S., Tumer, K.: Evolution-guided policy gradient in reinforcement learning. In: *Proceedings of the 32nd International Conference on Neural Information Processing Systems*, pp. 1196–1208 (2018)
14. Lee, K., Lee, B.U., Shin, U., Kweon, I.S.: An efficient asynchronous method for integrating evolutionary and gradient-based policy search. arXiv preprint [arXiv:2012.05417](https://arxiv.org/abs/2012.05417) (2020)
15. Lillicrap, T.P., et al.: Continuous control with deep reinforcement learning. arXiv preprint [arXiv:1509.02971](https://arxiv.org/abs/1509.02971) (2015)
16. Majid, A.Y., Saaybi, S., van Rietbergen, T., Francois-Lavet, V., Prasad, R.V., Verhoeven, C.: Deep reinforcement learning versus evolution strategies: a comparative survey. arXiv preprint [arXiv:2110.01411](https://arxiv.org/abs/2110.01411) (2021)
17. Marchesini, E., Corsi, D., Farinelli, A.: Genetic soft updates for policy evolution in deep reinforcement learning. In: *International Conference on Learning Representations* (2021)

18. Mitchell, M.: An Introduction to Genetic Algorithms. MIT Press, Cambridge (1998)
19. Mnih, V., et al.: Playing atari with deep reinforcement learning. arXiv preprint [arXiv:1312.5602](https://arxiv.org/abs/1312.5602) (2013)
20. Nair, A., McGrew, B., Andrychowicz, M., Zaremba, W., Abbeel, P.: Overcoming exploration in reinforcement learning with demonstrations. In: 2018 IEEE International Conference on Robotics and Automation (ICRA), pp. 6292–6299. IEEE (2018)
21. Pierrot, T., et al.: Diversity policy gradient for sample efficient quality-diversity optimization. In: ICLR Workshop on Agent Learning in Open-Endedness (2022)
22. Pourchot, A., Sigaud, O.: CEM-RL: combining evolutionary and gradient-based methods for policy search. In: International Conference on Learning Representations (2019)
23. Qian, H., Yu, Y.: Derivative-free reinforcement learning: a review. arXiv preprint [arXiv:2102.05710](https://arxiv.org/abs/2102.05710) (2021)
24. Salimans, T., Ho, J., Chen, X., Sidor, S., Sutskever, I.: Evolution strategies as a scalable alternative to reinforcement learning. arXiv preprint [arXiv:1703.03864](https://arxiv.org/abs/1703.03864) (2017)
25. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O.: Proximal policy optimization algorithms. arXiv preprint [arXiv:1707.06347](https://arxiv.org/abs/1707.06347) (2017)
26. Fujimoto, S.: Open-source implementation for TD3. <https://github.com/sfujim/TD3> (2018)
27. Sigaud, O.: Combining evolution and deep reinforcement learning for policy search: a survey. arXiv preprint [arXiv:2203.14009](https://arxiv.org/abs/2203.14009) (2022)
28. Silver, D., et al.: Mastering the game of go with deep neural networks and tree search. *Nature* **529**(7587), 484–489 (2016)
29. Silver, D., Lever, G., Heess, N., Degris, T., Wierstra, D., Riedmiller, M.: Deterministic policy gradient algorithms. In: International Conference on Machine Learning, pp. 387–395. PMLR (2014)
30. Such, F.P., Madhavan, V., Conti, E., Lehman, J., Stanley, K.O., Clune, J.: Deep neuroevolution: genetic algorithms are a competitive alternative for training deep neural networks for reinforcement learning. arXiv preprint [arXiv:1712.06567](https://arxiv.org/abs/1712.06567) (2017)
31. Todorov, E., Erez, T., Tassa, Y.: MuJoCo: a physics engine for model-based control. In: 2012 IEEE/RSJ International Conference on Intelligent Robots and Systems, pp. 5026–5033. IEEE (2012)
32. Wang, D., Tan, D., Liu, L.: Particle swarm optimization algorithm: an overview. *Soft. Comput.* **22**(2), 387–408 (2018)